

Apfluence — System Reference

Last updated: 2026-04-14 **Purpose:** Pre-campaign build scan. Living reference for all future feature builds. **Rule:** Every new agent or session reads this file first. Do not start writing code until this file is understood.

1. Database Schema

1.1 Live Migration Files (Source of Truth)

File	Status
supabase/migrations/001_Initial_Bdd_schema.sql	✓ LIVE — primary schema
supabase/migrations/002_seed_influencers.sql (or similar)	✓ LIVE — influencer seed
supabase/migrations/003_brand_lists.sql	✓ LIVE — lists & many-to-many
supabase/schema.sql	⚠ STALE DRAFT — ignore entirely
types/supabase.ts	⚠ SCHEMA DRIFT — stale types, use with caution

⚠ **CRITICAL:** `supabase/schema.sql` contains phantom tables (`campaigns` , `deals` , `users`) that **do not exist** in the live database. Always refer to the numbered migration files only.

1.2 All Live Tables

platforms

Column	Type	Notes
id	UUID PK	
name	VARCHAR	'instagram' \ 'tiktok' \ 'youtube' \ 'x' \ 'twitch'
display_name	VARCHAR	Human-readable
icon_url	VARCHAR	nullable
created_at	TIMESTAMPTZ	

influencers

Column	Type	Notes
<code>id</code>	UUID PK	Real Supabase UUID used everywhere in the UI
<code>first_name</code>	VARCHAR(100)	
<code>last_name</code>	VARCHAR(100)	
<code>name</code>	VARCHAR(201)	computed-style full name
<code>username</code>	VARCHAR(50)	unique
<code>email</code>	VARCHAR(320)	nullable — exists and is populated in seed
<code>avatar_url</code>	TEXT	nullable
<code>bio</code>	TEXT	nullable
<code>city</code>	VARCHAR(100)	nullable
<code>country_code</code>	CHAR(2)	ISO 3166-1 alpha-2
<code>flag</code>	VARCHAR(10)	emoji flag
<code>niches</code>	TEXT[]	categories collapsed here (no separate <code>categories</code> column)
<code>gallery</code>	TEXT[]	array of image URLs
<code>authenticity_score</code>	INTEGER	nullable
<code>created_at</code>	TIMESTAMPTZ	

⚠️ `categories` column does NOT exist. The `searchByTokens` function in `influencers.ts` has a bug that tries to filter on `categories` — it will silently fail. All category data lives in `niches`.

`influencer_platform_metrics`

Column	Type	Notes
<code>id</code>	UUID PK	
<code>influencer_id</code>	UUID FK → <code>influencers.id</code>	
<code>platform_id</code>	UUID FK → <code>platforms.id</code>	
<code>handle</code>	VARCHAR(100)	
<code>followers</code>	INTEGER	
<code>engagement_rate</code>	NUMERIC(5,2)	e.g. <code>10.80</code>
<code>avg_likes</code>	INTEGER	nullable
<code>avg_comments</code>	INTEGER	nullable
<code>avg_shares</code>	INTEGER	nullable (TikTok only)
<code>avg_views</code>	INTEGER	nullable (YouTube/TikTok)
<code>est_price_min</code>	INTEGER	in cents, nullable
<code>est_price_max</code>	INTEGER	in cents, nullable
<code>cpe</code>	NUMERIC	nullable
<code>cpv</code>	NUMERIC	nullable
<code>created_at</code>	TIMESTAMPTZ	

`influencer_posts`

Column	Type	Notes
<code>id</code>	UUID PK	
<code>influencer_id</code>	UUID FK → <code>influencers.id</code>	
<code>platform_id</code>	UUID FK → <code>platforms.id</code>	
<code>thumbnail_url</code>	TEXT	
<code>caption</code>	TEXT	
<code>likes</code>	INTEGER	
<code>comments</code>	INTEGER	
<code>views</code>	INTEGER	nullable
<code>shares</code>	INTEGER	nullable
<code>engagement_rate</code>	NUMERIC(5,2)	
<code>estimated_impressions</code>	INTEGER	nullable
<code>estimated_reach</code>	INTEGER	nullable
<code>cpe</code>	NUMERIC	nullable
<code>posted_at</code>	TIMESTAMPTZ	

brands

Column	Type	Notes
<code>id</code>	UUID PK	brand's internal Supabase UUID
<code>clerk_id</code>	TEXT UNIQUE	Clerk <code>userId</code> — the join key for all auth
<code>company_name</code>	VARCHAR(200)	
<code>logo_url</code>	TEXT	nullable — Cloudinary URL
<code>website</code>	TEXT	nullable
<code>industry</code>	VARCHAR(100)	nullable
<code>created_at</code>	TIMESTAMPTZ	
<code>updated_at</code>	TIMESTAMPTZ	

⚠ `types/supabase.ts` has `brands.user_id` instead of `brands.clerk_id` — this is a stale type. The real column in the DB is `clerk_id`.

`brand_lists`

Column	Type	Notes
<code>id</code>	UUID PK	
<code>brand_id</code>	UUID FK → <code>brands.id</code>	
<code>name</code>	VARCHAR(100)	
<code>description</code>	TEXT	nullable
<code>color</code>	VARCHAR(7)	hex string e.g. <code>#8B5CF6</code> , nullable
<code>created_at</code>	TIMESTAMPTZ	
<code>updated_at</code>	TIMESTAMPTZ	

`brand_list_influencers` (junction)

Column	Type	Notes
<code>id</code>	UUID PK	
<code>list_id</code>	UUID FK → <code>brand_lists.id</code>	CASCADE DELETE
<code>influencer_id</code>	UUID FK → <code>influencers.id</code>	CASCADE DELETE
<code>saved_at</code>	TIMESTAMPTZ	
UNIQUE	<code>(list_id, influencer_id)</code>	no duplicates

`saved_influencers`

Column	Type	Notes
<code>id</code>	UUID PK	
<code>brand_id</code>	UUID FK → <code>brands.id</code>	
<code>influencer_id</code>	UUID FK → <code>influencers.id</code>	
<code>status</code>	VARCHAR	'saved' \ 'rejected' \ 'contacted' (no FK constraint)
<code>notes</code>	TEXT	nullable
<code>saved_at</code>	TIMESTAMPTZ	

Note: `saved_influencers` is a separate table from `brand_list_influencers`. Being in a list ≠ being "saved" here.

1.3 RLS Status

Table	RLS	Policies
<code>influencers</code>	Enabled	<code>FOR SELECT USING (true)</code> — public read
<code>influencer_platform_metrics</code>	Enabled	<code>FOR SELECT USING (true)</code> — public read
<code>influencer_posts</code>	Enabled	<code>FOR SELECT USING (true)</code> — public read
<code>platforms</code>	Enabled	<code>FOR SELECT USING (true)</code> — public read
<code>brands</code>	Enabled	policy exists but dormant
<code>brand_lists</code>	Enabled	policy exists but dormant
<code>brand_list_influencers</code>	Enabled	policy exists but dormant

⚠ **RLS is effectively dormant for brand data.** The app exclusively uses `supabaseAdmin` (service role key) which bypasses all RLS. Security is enforced at the application layer by manually filtering on `brand_id`. RLS policies exist but are never the active enforcement layer.

1.4 What Does NOT Exist in the Live DB

- ✗ `campaigns` (phantom in `schema.sql` — not real)
- ✗ `campaign_influencers` (not real)
- ✗ `campaign_deliverables` (not real)
- ✗ `deals` (phantom in `schema.sql` — not real)
- ✗ `users` (phantom in `types/supabase.ts` — not real)
- ✗ `brands.user_id` (stale type — real column is `brands.clerk_id`)
- ✗ `influencers.categories` (collapsed into `influencers.niches`)

1.5 Schema Summary

TABLE	PK	FK TARGETS
platforms	UUID	–
influencers	UUID	–
influencer_platform_metrics	UUID	influencers, platforms
influencer_posts	UUID	influencers, platforms
brands	UUID	– (clerk_id = Clerk)
brand_lists	UUID	brands
brand_list_influencers	UUID	brand_lists, influencers
saved_influencers	UUID	brands, influencers
RLS STATUS: Enabled on all tables		
ENFORCEMENT: supabaseAdmin (service role) bypasses RLS		
SECURITY: manual brand_id filter in every server action		
CAMPAIGNS TABLES: ❌ DO NOT EXIST – must be created		

2. Authentication & Session

2.1 Identity Provider

Clerk — custom UI (not Clerk's prebuilt UI). Roles stored in Clerk `publicMetadata.role`.

2.2 How Auth Works End to End

1. User signs up → custom `/sign-up/brand` page
2. Brand fills company details → `POST /api/set-role`
3. `/api/set-role`:
 - a. Sets clerk `publicMetadata.role = "brand"`
 - b. INSERTs row into brands (clerk_id, company_name, etc.) via `supabaseAdmin`
4. Clerk JWT now contains role in `publicMetadata`
5. `middleware.ts` reads role from JWT → routes accordingly
6. All `/brand/*` pages protected by 3-layer auth check

2.3 Three-Layer Auth Protection

```
Layer 1: middleware.ts (Edge)
└─ clerkMiddleware()
    └─ Unauthenticated → /sign-in/brand
    └─ role ≡ "brand" → allow /brand/*
    └─ role ≡ "influencer" → redirect to /influencer/*

Layer 2: app/(protected)/layout.tsx (Server Component)
└─ const { userId } = await auth()
    └─ !userId → redirect("/sign-in/brand")

Layer 3: app/(protected)/brand/layout.tsx (Server Component)
└─ const { userId, sessionClaims } = await auth()
└─ !userId → redirect("/sign-in/brand")
└─ Stale JWT Recovery: re-fetch from Clerk if role missing
└─ role ≡ "brand" → redirect to /sign-up/brand or /influencer
└─ renders: <MobileBlocker /> + <Sidebar /> + {children}
```

2.4 How `clerk_id` Feeds Data Access

There is NO RLS feeding from Clerk. The pattern is always:

```
// 1. Get Clerk userId
const { userId } = await auth(); // server
const { user } = useUser(); // client → user.id

// 2. Resolve brand UUID from clerk_id
const brandId = await getBrandId(userId);
// getBrandId = supabaseAdmin.from('brands').select('id').eq('clerk_id', userId)

// 3. Query with brand_id filter (NOT clerk_id)
const data = await supabaseAdmin
  .from("brand_lists")
  .select("*")
  .eq("brand_id", brandId);
```

2.5 Exact Code Patterns — See Section 9 for full examples

2.6 How Brand Row Is Inserted

```
// app/api/set-role/route.ts — called during sign-up
await supabaseAdmin.from("brands").insert({
  clerk_id: userId,
  company_name: companyName,
  logo_url: logoUrl ?? null,
  website: website ?? null,
  industry: industry ?? null,
});
await clerkClient.users.updateUserMetadata(userId, {
  publicMetadata: { role },
});
```

2.7 Roles

Role	Value	Where Stored
Brand	"brand"	Clerk <code>publicMetadata.role</code>
Influencer	"influencer"	Clerk <code>publicMetadata.role</code>

2.8 Auth Summary

AUTH SUMMARY		
Identity provider	:	Clerk (custom UI)
Role storage	:	Clerk publicMetadata.role
Roles	:	"brand" "influencer"
Server clerkId	:	await auth() → userId
Client clerkId	:	useUser() → user.id
Brand UUID resolution	:	getBrandId(clerkId)
	→	brands.select('id').eq('clerk_id', clerkId)
DB access method	:	supabaseAdmin everywhere
RLS active	:	NO (bypassed by admin key)
Security enforcement	:	manual .eq('brand_id', id)
Brand row created	:	POST /api/set-role
Logo storage	:	Cloudinary (not Supabase)
Mobile access	:	blocked by MobileBlocker

3. Lists System

3.1 File Map

File	Purpose
<code>lib/queries/lists.ts</code>	Server actions — all DB ops via <code>supabaseAdmin</code>
<code>lib/queries/lists.client.ts</code>	Client wrapper — in-memory cache + invalidation
<code>lib/cache/listsCache.ts</code>	<code>SimpleCache</code> class — Map-based, 30s TTL
<code>components/brand/lists/CreateListModal.tsx</code>	Modal: create list with name + color
<code>components/brand/lists/ListsDropdown.tsx</code>	Dropdown: view/delete lists
<code>components/brand/sidebar/SaveToListButton.tsx</code>	Toggle: add/remove influencer from list

3.2 `BrandList` Shape (What the UI Uses)

```
// Full DB row shape (from server query):
type BrandListWithCount = {
  id: string; // UUID
  brand_id: string; // UUID → brands.id
  name: string;
  description: string | null;
  color: string | null; // hex e.g. "#8B5CF6"
  created_at: string;
  updated_at: string;
  influencer_count: number; // computed — NOT a real DB column
};

// Minimal shape used in ListsDropdown UI (local interface):
interface BrandList {
  id: string;
  name: string;
  color: string | null;
  influencer_count: number;
  created_at: string;
}
```

3.3 How `influencer_count` Is Computed

```
// In lists.ts – single Supabase aggregate query:
supabaseAdmin
  .from("brand_lists")
  .select("*, brand_list_influencers(count)")
  .eq("brand_id", brandId);

// Mapped to:
influencer_count: list.brand_list_influencers?.[0]?.count ?? 0;
// Supabase returns [{ count: N }] for aggregate shorthand
```

3.4 Cache Architecture

```
// lib/cache/listsCache.ts
class SimpleCache {
  set<T>(key: string, data: T, ttlMs = 30_000): void;
  get<T>(key: string): T | null; // returns null if expired
  invalidate(key: string): void;
  invalidatePrefix(prefix: string): void; // nukes all keys starting with prefix
}
export const listsCache = new SimpleCache();

// Cache keys:
// "lists:{clerkId}"           – plain lists array
// "lists_with_first:{clerkId}" – lists + first list's influencers

// Invalidation: ALL 4 mutating functions call:
listsCache.invalidatePrefix("lists"); // wipes both keys
```

3.5 Known Bugs / TODOs

#	Location	Issue
1	<code>influencers.ts</code>	<code>searchByTokens</code> filters on <code>categories</code> column which does NOT exist — will silently fail
2	<code>listsCache.ts</code>	Module-level singleton — not shared across server instances in production edge
3	<code>lists.client.ts</code>	No <code>'use client'</code> directive — naming implies client but internally calls server actions
4	<code>CreateListModal.tsx</code>	<code>influencer_count: 0</code> injected locally after creation — not refreshed from server
5	<code>ListsDropdown.tsx</code>	No <code>onClick</code> routing on list items — dropdown is display-only

3.6 Lists Summary

```
|  LISTS SYSTEM SUMMARY  |
|  Server query file      : lists.ts      |
|  Client query file     : lists.client.ts |
|  Cache TTL              : 30,000ms (30s) |
|  influencer_count method :              |
|    SELECT *, brand_list_influencers(count) |
|    → list.brand_list_influencers[0].count |
|  Cache invalidation    : YES            |
|    invalidatePrefix('lists') on all writes |
|  Active lists UI page   : /brand/selected |
|    (/brand/lists route = ComingSoon stub) |
```

4. Influencer Data Layer

4.1 File Map

File	Purpose
lib/queries/influencers.ts	All DB queries → returns <code>NormalizedInfluencer[]</code>
lib/adapters/normalizeInfluencer.ts	<code>InfluencerWithDetails</code> → <code>NormalizedInfluencer</code>
components/brand/sidepanel/influencer-side-panel.types.ts	All type definitions
lib/data.ts	⚠ Legacy mock data — still on disk, no longer imported

4.2 Full `NormalizedInfluencer` Type


```
export interface NormalizedInfluencer {
  id: string; // ← real influencers.id UUID – use this everywhere
  name: string;
  firstName: string;
  lastName: string;
  username: string;
  email: string | null; // EXISTS – maps from influencers.email VARCHAR(320)
  avatar: string | null; // maps from avatar_url
  bio: string | null;
  authenticityScore: number | null;
  location: {
    city: string | null;
    countryCode: string | null;
    flag: string | null;
  };
  niches: string[];
  categories: string[]; // ⚠️ ALIAS – maps to niches[] (categories was collapsed)
  gallery: string[];
  metrics: NormalizedMetric[]; // from influencer_platform_metrics JOIN platforms
  posts: NormalizedPost[]; // from influencer_posts JOIN platforms
}

interface NormalizedMetric {
  platform: "instagram" | "tiktok" | "youtube" | "x" | "twitch";
  platform_display_name: string;
  handle: string;
  followers: number;
  engagement: number; // e.g. 10.80
  avgLikes: number | null;
  avgComments: number | null;
  avgShares: number | null;
  avgViews: number | null;
  estimatedPriceMin: number | null; // in cents
  estimatedPriceMax: number | null; // in cents
  cpe: number | null;
  cpv: number | null;
  estimatedPriceRange?: string | null; // formatted: "$1.2k - $2.8k"
}

interface NormalizedPost {
  id: string;
  platform: PlatformName;
  thumbnailUrl: string;
  caption: string;
  likes: number;
  comments: number;
  views: number | null;
```

```
shares: number | null;  
estimatedImpressions: number | null;  
estimatedReach: number | null;  
engagementRate: number;  
cpe: number | null;  
postedAt: string; // relative: "2 days ago"  
}
```

4.3 Main Supabase Query Pattern

All 5 query functions in `influencers.ts` use one SELECT — **single round trip**:

```
// Uses ANON KEY browser client (NOT supabaseAdmin)
// Safe because influencers table has: FOR SELECT USING (true)
supabase.from("influencers").select(`
  *,
  influencer_platform_metrics (*, platforms(name, display_name)),
  influencer_posts            (*, platforms(name, display_name))
`);
```

⚠ If `influencers` RLS is ever tightened, this will break. Consider switching to `supabaseAdmin` for resilience.

4.4 Normalizer Features

- **WeakMap memoization** — same raw object reference → cached normalized result
- `categories` alias — `categories: raw.niches ?? []` to prevent UI crashes
- **Formatters exported:** `formatFollowers(n)`, `formatPriceRange(min, max)`
- **Relative time:** `formatRelativeTime(timestamp)` → `"2 days ago"`

4.5 Influencer Data Summary

```
| INFLUENCER DATA SUMMARY |
| NormalizedInfluencer.id : |
|   influencers.id (UUID PRIMARY KEY) |
|   - pass this as influencerId everywhere |
| Email exists           : YES (nullable) |
|   influencers.email VARCHAR(320) |
|   → NormalizedInfluencer.email |
| Full SELECT           : 1 round-trip |
|   influencers.* |
|   + influencer_platform_metrics(*) |
|   + influencer_posts(*) |
| DB client used       : ANON KEY |
|   (NOT supabaseAdmin - influencers public) |
| Normalizer cache     : WeakMap |
```

5. Routing & Pages

5.1 All Routes Under `/brand/`

Route	Status	What It Renders
<code>/brand</code>	✓ LIVE	Discovery — <code>BrandHomeClient</code> with token search + influencer cards
<code>/brand/selected</code>	✓ LIVE	Lists sidebar + influencer grid (the real "saved to list" UI)
<code>/brand/campaigns</code>	⚠ PLACEHOLDER	<code>ComingSoonPage</code> — orange, 32%, "In active development"
<code>/brand/lists</code>	⚠ PLACEHOLDER	<code>ComingSoonPage</code> — indigo, 18%, "Coming Q3 2026"
<code>/brand/outreach</code>	? UNKNOWN	Assumed ComingSoon
<code>/brand/analytics</code>	? UNKNOWN	Assumed ComingSoon
<code>/brand/payments</code>	? UNKNOWN	Assumed ComingSoon
<code>/brand/settings</code>	? UNKNOWN	Assumed ComingSoon
<code>/brand/support</code>	? UNKNOWN	Assumed ComingSoon

⚠ **Naming confusion:** The Sidebar item "Lists" links to `/brand/lists` (a placeholder). The **working** list management UI is at `/brand/selected`. These are two different things.

5.2 Sidebar Navigation

File: `components/brand/Sidebar.tsx` — fixed `w-14` (56px), `z-[60]`, icon-only

```
const NAV = [  
  { name: "Dashboard", href: "/brand", icon: LayoutDashboard },  
  { name: "Lists", href: "/brand/lists", icon: List },  
  { name: "Discovery", href: "/brand/discovery", icon: Search },  
  { name: "Outreach", href: "/brand/outreach", icon: Mail },  
  { name: "Payments", href: "/brand/payments", icon: CreditCard },  
  { name: "Analytics", href: "/brand/analytics", icon: BarChart3 },  
  { name: "Campaigns", href: "/brand/campaigns", iconSrc: "/campaigns.png" }, // - PNG, typo intentional  
  { name: "Support", href: "/brand/support", icon: Headphones },  
];  
  
// Active detection:  
function isActive(pathname, href) {  
  if (href === "/brand") return pathname === "/brand" || pathname === "/brand/";  
  return pathname === href || pathname.startsWith(`${href}/`);  
}  
  
// Active indicator: motion.div layoutId="sidebar-active" - animated 3px white bar
```

⚠️ `/campaings.png` — the filename has a typo ("campaings" not "campaigns"). **Do not rename it**, it is referenced in both the Sidebar and the campaigns `ComingSoonPage`.

5.3 Header Navigation

File: `components/brand/Header.tsx` — fixed `h-[52px]`, offset `left-14`, `z-50`

```
const tabs = [
  { label: "Find creators", badge: null },
  { label: "Selected",      badge: selectedCount, badgeColor: "bg-[var(--color-success)]" },
  { label: "Rejected",      badge: 0 },      // ← STUB, no route
  { label: "Suggested",     badge: 0 },      // ← STUB, no route
]

// Tab click → router.push():
"Find creators" → /brand
"Selected"      → /brand/selected
"Rejected"      → setFallbackTab() only (no navigation)
"Suggested"     → setFallbackTab() only (no navigation)

// Active tab detection:
const activeTab = useMemo(() => {
  if (pathname === "/brand/selected") return "Selected"
  if (pathname === "/brand" || pathname === "/brand/discovery") return "Find creators"
  return fallbackTab
}, [pathname, fallbackTab])

// Active indicator: motion.div layoutId="header-tab" — animated 2px underline
```

Left side: `ListsDropdown` — Right side: Learn + `UserButton` (Clerk)

5.4 Routing Summary

```
| ROUTING SUMMARY |
| Brand routes:   |
|   /brand        LIVE (Discovery) |
|   /brand/selected LIVE (Lists UI) |
|   /brand/campaigns PLACEHOLDER 32% |
|   /brand/lists   PLACEHOLDER 18% |
|   others         UNKNOWN           |
| Tab nav method  : router.push()    |
| Active tab      : usePathname() match |
| Auth protection : 3-layer (middleware |
|   + (protected)/layout + brand/layout |
| Mobile          : blocked (MobileBlocker) |
```


6. Component Inventory

6.1 Component Details

Component	File	Purpose	Reuse for Campaigns
InfluencerResultCard	components/brand/InfluencerResultCard.tsx	Horizontal card: avatar, bio, metrics, niches. Clickable → side panel	✅ Direct reuse
InfluencerSidePanel	components/brand/InfluencerSidePanel.tsx	Slide-out sheet: channels, email, notes, profile tabs	✅ Direct reuse
Header	components/brand/Header.tsx	Fixed top bar: tabs + lists dropdown + user	✅ Direct reuse
Sidebar	components/brand/Sidebar.tsx	Fixed left nav: 8 icon items	✅ Direct reuse (already has Campaigns link)
SaveToListButton	components/brand/sidepanel/SaveToListButton.tsx	Toggle add/remove influencer from list	⚠️ Adapt → "Add to Campaign"
ListsDropdown	components/brand/lists/ListsDropdown.tsx	Dropdown: view/delete lists	⚠️ Inspire pattern
CreateListModal	components/brand/lists/CreateListModal.tsx	Modal: create list (name + color)	⚠️ Inspire → "Create Campaign" modal
SearchSection	components/brand/SearchSection.tsx	Discovery search row wrapper	❌ Discovery-only
TokenSearchBar	components/brand/TokenSearchBar.tsx	Multi-token pill input	⚠️ Maybe (handle search)

Component	File	Purpose	Reuse for Campaigns
<code>ImportSection</code>	<code>components/brand/ImportSection.tsx</code>	Chrome plugin + CSV upload CTAs	✗ Discovery onboarding only
<code>ComingSoonPage</code>	<code>components/brand/coming-soon/ComingSoonPage.tsx</code>	Animated placeholder for unbuilt pages	✗ IS the current campaigns page — replace it
<code>BrandHomeClient</code>	<code>components/brand/BrandHomeClient.tsx</code>	Discovery page orchestrator	✗ Discovery-specific

6.2 Known Component Issues

Component	Issue
<code>InfluencerResultCard</code>	Accept/Reject buttons (✓/✗) have zero logic — pure UI stubs
<code>InfluencerSidePanel</code>	"Email", "Notes", "Profile" tabs contain literal placeholder strings
<code>SaveToListButton</code>	<code>influencerId</code> must be real Supabase UUID — not username

6.3 No Shared Primitives

- **Empty state component:** ✗ None — inline JSX per page
- **Loading skeleton:** ✗ None — use `<Loader2 className="animate-spin text-violet-400" />` inline
- **Generic modal:** ✗ None — model after `CreateListModal` pattern

6.4 Animation Library

Import style	Used in	When to use
<code>import { LazyMotion, domAnimation, m } from framer-motion</code>	Modals, dropdowns, <code>ComingSoonPage</code>	Preferred — smaller bundle
<code>import { motion } from framer-motion</code>	<code>InfluencerResultCard</code>	Simple card hover
<code>import { motion } from motion/react</code>	<code>Sidebar</code> , <code>Header</code>	Same as above

For new campaigns code: use `LazyMotion + domAnimation + m`.

6.5 UI Library

shadcn/ui — `components/ui/` — uses Radix UI primitives internally. Available: `Sheet`, `Tabs`, `TabsContent`, `Button`, `Input`, `Label`, `InputOTP`, `Dialog`, `Progress`, `Select`, `Checkbox`, `Slider`, `Switch`, `Tooltip`, and more.

7. Environment & Config

7.1 Core Versions

Item	Version
Next.js	16.2.2 (App Router, fully adopted)
React	19.2.4
TypeScript	^5 (strict mode ON)
Tailwind CSS	^4
Package manager	pnpm

7.2 Key Dependencies

Category	Package	Version	Status
Auth	@clerk/nextjs	^7.0.11	✓
Database	@supabase/supabase-js	^2.103.0	✓
Database SSR	@supabase/ssr	^0.10.2	✓
Animation	framer-motion	^12.38.0	✓
Animation	motion	12.23.24	✓
Forms	react-hook-form	7.55.0	✓
Dates	date-fns	3.6.0	✓
Date picker	react-day-picker	8.10.1	✓
Drag & drop	react-dnd + react-dnd-html5-backend	16.0.1	✓
Charts	recharts	2.15.2	✓
Toasts	sonner	2.0.3	✓
Icons	lucide-react	0.487.0	✓
Schema validation	zod	—	✗ NOT INSTALLED
Email	resend / nodemailer / sendgrid	—	✗ NOT INSTALLED
Email templates	@react-email/*	—	✗ NOT INSTALLED
Cron / queues	bull / qstash	—	✗ NOT INSTALLED

7.3 next.config.ts Key Points

```
experimental: { optimizePackageImports: ["lucide-react", "framer-motion"] }
compress: true
images.remotePatterns: [unsplash.com, api.dicebear.com, assets.aceternity.com]
headers: Cache-Control: private, no-cache on /brand/*
```

⚠️ `res.cloudinary.com` missing from `remotePatterns` — brand logo URLs from Cloudinary will fail through `next/image`. Existing bug — fix before campaigns.

7.4 Environment Variables (Names Only)

Variable	Scope	Purpose
<code>NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY</code>	Public	Clerk client
<code>CLERK_SECRET_KEY</code>	Server	Clerk server API
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Public	Supabase project URL
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Public	Supabase public key
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Server	Admin key — bypasses RLS
<code>NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME</code>	Public	Cloudinary cloud
<code>NEXT_PUBLIC_CLOUDINARY_UPLOAD_PRESET</code>	Public	Cloudinary upload preset

Vars to add for campaigns: | Variable | Purpose | |---|---| | `RESEND_API_KEY` | Email sending (if added) | | `NEXT_PUBLIC_APP_URL` | Email template callback links |

7.5 Config Summary

```

| CONFIG SUMMARY |
| Next.js       : 16.2.2 (App Router) |
| React         : 19.2.4               |
| Package mgr   : pnpm                 |
| TS strict     : ON                   |
| Email pkg     : ❌ NONE                |
| Validation pkg : ❌ NONE (no zod/yup) |
| Animation     : framer-motion ^12.38.0 |
| Dates         : date-fns + react-day-picker |
| Drag & drop    : react-dnd ✅         |
| Charts        : recharts ✅          |
| Toasts        : sonner ✅            |
| Missing:      |
|   zod + @hookform/resolvers          |
|   resend + @react-email/components   |
|   res.cloudinary.com in remotePatterns |

```

8. Campaign Build Readiness

8A — Reusable Assets (Use These Directly)

Asset	Where	How to Use
<code>InfluencerResultCard</code>	<code>components/brand/</code>	Show influencers inside campaign detail view
<code>InfluencerSidePanel</code>	<code>components/brand/</code>	Inspect influencer profiles from campaign view
<code>Header</code>	<code>components/brand/</code>	Import as-is on campaigns page
<code>Sidebar</code>	<code>components/brand/</code>	Already present via <code>brand/layout.tsx</code>
<code>getBrandId(clerkId)</code>	<code>lib/queries/lists.ts</code>	Always resolve brand UUID before DB ops
<code>supabaseAdmin</code>	<code>lib/supabase/admin.ts</code>	All server-side DB access
<code>sonner</code> toast	Installed	Use for success/error feedback on campaign actions
<code>date-fns</code>	Installed	Format campaign dates, compute deadlines
<code>react-day-picker</code>	Installed	Date inputs for campaign start/end dates
<code>react-dnd</code>	Installed	Kanban-style influencer reordering
<code>recharts</code>	Installed	Budget/performance dashboards
<code>react-hook-form</code>	Installed	Campaign create/edit forms
shadcn/ui <code>Dialog</code>	<code>components/ui/</code>	"Create Campaign" modal
shadcn/ui <code>Progress</code>	<code>components/ui/</code>	Budget progress bars
shadcn/ui <code>Select</code>	<code>components/ui/</code>	Status dropdowns
shadcn/ui <code>Tabs</code>	<code>components/ui/</code>	Campaign detail tabs

8B — What Must Be Built from Scratch

Database Layer:

- [] Migration: `campaigns` table (`id` , `brand_id` , `name` , `description` , `status` , `budget` , `start_date` , `end_date` , `created_at`)
- [] Migration: `campaign_influencers` (`id` , `campaign_id` , `influencer_id` , `status` , `fee` , `added_at`)
- [] Migration: `campaign_deliverables` (`id` , `campaign_influencer_id` , `title` , `due_date` , `status` , `notes`)
- [] Update `types/supabase.ts` after migrations run

Service Layer:

- [] `lib/queries/campaigns.ts` — server actions (getCampaigns, createCampaign, addInfluencerToCampaign, etc.)
- [] `lib/queries/campaigns.client.ts` — client wrapper with cache (follow `lists.client.ts` pattern)
- [] `lib/cache/campaignsCache.ts` — follow `listsCache.ts` pattern exactly

UI Layer:

- [] `app/(protected)/brand/campaigns/page.tsx` — replace `ComingSoonPage` with real dashboard
- [] `components/brand/campaigns/CampaignCard.tsx` — campaign summary card
- [] `components/brand/campaigns/CreateCampaignModal.tsx` — modal (model after `CreateListModal`)
- [] `components/brand/campaigns/CampaignDetailView.tsx` — influencer pipeline + deliverables

Optional (Phase 2):

- [] Install `zod` + `@hookform/resolvers` for form validation
- [] Install `resend` + `@react-email/components` for email briefing

8C — Breaking Risks

Risk	Severity	Details
<code>types/supabase.ts</code> drift	● Medium	Stale types (<code>brands.user_id</code> vs <code>brands.clerk_id</code>). New campaign types won't exist until regenerated after migration. Write types manually until then.
<code>categories</code> filter bug	● Medium	<code>searchByTokens</code> filters on non-existent <code>categories</code> column. Fix before campaigns if search is needed inside a campaign.
Cloudinary NOT in <code>remotePatterns</code>	● Medium	Brand <code>logo_url</code> won't render in campaigns if using <code>next/image</code> . Fix <code>next.config.ts</code> first.
<code>supabaseAdmin</code> required always	● Low	All campaign server actions must use <code>supabaseAdmin</code> , never the anon client. Already the pattern — just don't deviate.
Cache not shared across edge workers	● Low	<code>campaignsCache</code> (like <code>listsCache</code>) will not share state across multiple server processes in production. Acceptable for MVP.
pnpm lockfile	● Low	Do NOT run <code>npm install</code> — always use <code>pnpm add</code> .

8D — Recommended Build Order

Step	Action	Rationale
1	Fix <code>next.config.ts</code> — add <code>res.cloudinary.com</code> to <code>remotePatterns</code>	Unblocks brand logo display everywhere
2	Write + apply <code>004_campaigns.sql</code> migration	Creates the 3 campaign tables
3	Manually write campaign types in <code>types/supabase.ts</code>	Can't run type-gen until Supabase CLI is configured
4	Build <code>lib/queries/campaigns.ts</code> server actions	Follow <code>lists.ts</code> patterns exactly
5	Build <code>lib/queries/campaigns.client.ts</code> + <code>campaignsCache.ts</code>	Follow <code>lists.client.ts</code> + <code>listsCache.ts</code> exactly
6	Build <code>CreateCampaignModal.tsx</code>	Follow <code>CreateListModal.tsx</code> exactly
7	Replace <code>campaigns/page.tsx</code> with real dashboard	Replace <code>ComingSoonPage</code> with campaign list
8	Build <code>CampaignDetailView</code> — influencer pipeline	Uses <code>InfluencerResultCard</code> + <code>InfluencerSidePanel</code>
9	(Optional) Install <code>zod</code> + <code>@hookform/resolvers</code>	For campaign form validation
10	(Optional) Install <code>resend</code> + <code>@react-email</code>	For email campaign briefs

8E — Open Questions

1. **Campaign status workflow** — what are the valid status values? (e.g. `draft` → `active` → `paused` → `completed`) — needs product decision
2. **Budget model** — flat budget per campaign, or per-influencer fee? Or both?
3. **Deliverable types** — what counts as a deliverable? (post, reel, story, video?) — affects `campaign_deliverables` schema
4. **Campaign ↔ Lists relationship** — can influencers be bulk-added from an existing `brand_list` to a campaign? This is a highly requested UX pattern.
5. **Multi-brand** — currently one brand per Clerk user. Will campaigns need to support agency/team access? If yes, this needs a `brand_members` table before campaigns.
6. **Influencer notification** — does the MVP need to notify influencers (email/in-app) when added to a campaign? If yes, install `resend` in Step 4, not Step 10.

8F — Readiness Block

```
| CAMPAIGN BUILD READINESS |
|                             |
| DB tables ready           : ❌ (must build) |
| Auth pattern ready       : ✅ |
| Service layer pattern    : ✅ (follow lists)|
| Cache pattern ready      : ✅ (follow lists)|
| Route exists             : ✅ /brand/campaigns |
| Current page             : ComingSoonPage |
| (replace entirely)      |
| Reusable UI components  : ✅ (card+panel) |
| Forms                   : ✅ (react-hook-form) |
| Dates                   : ✅ (date-fns) |
| Toasts                  : ✅ (sonner) |
| Email infra             : ❌ (not installed)|
| Zod validation           : ❌ (not installed)|
| Cloudinary remotePattern : ❌ (bug - fix first)|
|                             |
| FIRST ACTION: fix next.config.ts |
| SECOND ACTION: write 004_campaigns.sql |
```

9. Patterns & Conventions

These are the laws of this codebase. Follow them in every new file. Do not invent new patterns.

9.1 Supabase Server Query Pattern

```
// lib/queries/campaigns.ts
"use server";

import { supabaseAdmin } from "@lib/supabase/admin";

// Step 1: Always get brand UUID from clerk_id first
async function getBrandId(clerkId: string): Promise<string | null> {
  const { data } = await supabaseAdmin
    .from("brands")
    .select("id")
    .eq("clerk_id", clerkId)
    .single();
  return data?.id ?? null;
}

// Step 2: All queries use supabaseAdmin + filter by brand_id
export async function getCampaigns(clerkId: string) {
  const brandId = await getBrandId(clerkId);
  if (!brandId) return [];

  const { data, error } = await supabaseAdmin
    .from("campaigns")
    .select("*")
    .eq("brand_id", brandId)
    .order("created_at", { ascending: false });

  if (error) {
    console.error("[getCampaigns]", error);
    return [];
  }

  return data ?? [];
}
```

Rules:

- Always `supabaseAdmin` — never anon client for brand-owned data
- Always resolve `brand_id` via `getBrandId(clerkId)` before querying
- Never pass `brand_id` from the client — always resolve server-side
- Return empty array on error, never throw to the client

9.2 Supabase Client Query Pattern

```
// lib/queries/campaigns.client.ts
// NO 'use client' directive – this file is called FROM client components
// but itself calls 'use server' functions

import { campaignsCache } from '@lib/cache/campaignsCache'
import {
  getCampaigns as getCampaignsServer,
  createCampaign as createCampaignServer,
} from '@lib/queries/campaigns'

const TTL = 30_000 // 30 seconds

export async function getCampaigns(clerkId: string) {
  const key = `campaigns:${clerkId}`
  const cached = campaignsCache.get<Campaign[]>(key)
  if (cached) return cached

  const data = await getCampaignsServer(clerkId)
  campaignsCache.set(key, data, TTL)
  return data
}

export async function createCampaign(clerkId: string, name: string, ...) {
  const result = await createCampaignServer(clerkId, name, ...)
  if (result) campaignsCache.invalidatePrefix('campaigns')
  return result
}
```

9.3 How to Get clerkId in a Server Action

```
// In a Server Component or Server Action:
import { auth } from '@clerk/nextjs/server'

export async function CampaignPage() {
  const { userId } = await auth()
  if (!userId) redirect('/sign-in/brand')

  const campaigns = await getCampaigns(userId) // userId IS the clerkId
  ...
}
```

9.4 How to Get clerkId in a Client Component

```
'use client'
import { useUser } from '@clerk/nextjs'

export function CampaignClient() {
  const { user, isLoading } = useUser()

  useEffect(() => {
    if (!isLoading || !user?.id) return
    const clerkId = user.id // clerkId is user.id
    getCampaigns(clerkId).then(setData)
  }, [isLoading, user?.id])
  ...
}
```

9.5 How New Pages Are Structured

```
// app/(protected)/brand/campaigns/page.tsx

// OPTION A: Server Component (preferred for initial data)
import { auth } from '@clerk/nextjs/server'
import { redirect } from 'next/navigation'
import { getCampaigns } from '@lib/queries/campaigns'
import { CampaignClient } from '@components/brand/campaigns/CampaignClient'

export default async function CampaignsPage() {
  const { userId } = await auth()
  if (!userId) redirect('/sign-in/brand')

  const campaigns = await getCampaigns(userId)
  return <CampaignClient initialCampaigns={campaigns} clerkId={userId} />
}

// OPTION B: Client-only page (like /brand/selected)
'use client'
import { useUser } from '@clerk/nextjs'
import { Header } from '@components/brand/Header'
// ... rest of client logic
```

Note: Auth protection is already handled by `brand/layout.tsx`. The page itself does not need to re-check auth unless it needs `userId` for data fetching.

9.6 How New Components Are Organized

```
components/  
  brand/  
    campaigns/      ← NEW — mirrors: lists/, sidepanel/  
      CampaignCard.tsx  
      CreateCampaignModal.tsx  
      CampaignDetailView.tsx  
      CampaignInfluencerRow.tsx  
    lists/          ← existing  
    sidepanel/      ← existing
```

Rules:

- Feature components go in `components/brand/[feature]/`
- Types go in `[feature]/[feature].types.ts`
- Utils go in `[feature]/[feature].utils.ts`
- No inline large helper functions — extract to utils file

9.7 Naming Conventions

Entity	Convention	Example
Components	PascalCase	<code>CampaignCard.tsx</code>
Server query files	camelCase	<code>campaigns.ts</code>
Client query files	camelCase + <code>.client</code>	<code>campaigns.client.ts</code>
Cache files	camelCase + <code>Cache</code>	<code>campaignsCache.ts</code>
Type files	camelCase + <code>.types</code>	<code>campaigns.types.ts</code>
Util files	camelCase + <code>.utils</code>	<code>campaigns.utils.ts</code>
DB query functions	verb + noun	<code>getCampaigns</code> , <code>createCampaign</code>
Types	PascalCase	<code>Campaign</code> , <code>CampaignInfluencer</code>
CSS variables	<code>--color-[purpose]</code>	<code>--color-brand-blue</code>
Path alias	<code>@/</code> → root	<code>@/lib/queries/campaigns</code>

9.8 How Migrations Are Named and Structured

```
supabase/migrations/
  001_Initial_Bdd_schema.sql
  002_seed_influencers.sql   (or similar)
  003_brand_lists.sql
  004_campaigns.sql         ← next migration to write

-- Format inside migration file:
-- Each table in CREATE TABLE IF NOT EXISTS blocks
-- ALL FKs defined inline with REFERENCES
-- RLS: ENABLE then CREATE POLICY
-- Add COMMENT ON TABLE for documentation
-- End with a seed or no-op INSERT if needed for dev

-- Example:
CREATE TABLE IF NOT EXISTS campaigns (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  brand_id    UUID NOT NULL REFERENCES brands(id) ON DELETE CASCADE,
  name        VARCHAR(200) NOT NULL,
  status      VARCHAR(20) NOT NULL DEFAULT 'draft'
              CHECK (status IN ('draft','active','paused','completed')),
  budget      INTEGER,      -- in cents
  start_date  DATE,
  end_date    DATE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
ALTER TABLE campaigns ENABLE ROW LEVEL SECURITY;
```

10. What To Build Next

The **Campaigns feature** needs to give brands a structured command centre to manage influencer partnerships from brief to recap. The feature requires three new database tables (`campaigns` , `campaign_influencers` , `campaign_deliverables`), a full server query layer following the established `getBrandId` + `supabaseAdmin` pattern, a client cache layer following the `listsCache` + `lists.client.ts` pattern, and a real UI at `/brand/campaigns` replacing the current `ComingSoonPage` placeholder. The existing `InfluencerResultCard` , `InfluencerSidePanel` , `Header` , and `Sidebar` components can all be used directly without modification. `react-hook-form` , `date-fns` , `react-day-picker` , `react-dnd` , `recharts` , and `sonner` are already installed and ready. The **single most important thing to resolve before writing any campaign code** is the `next.config.ts` bug where `res.cloudinary.com` is missing from `remotePatterns` — brand logos will fail to render in the campaign UI without this fix, and it is a one-line change.

Document generated from complete reconnaissance scan — Steps 1–7 + Campaign Readiness Analysis. No source files were modified during the scan. This file is the single source of truth. Update it after every major feature build.